

reza-ansible

Step-by-step guide: provisioning a Debian VM with the preparing_server (hardening) role, and reusing this project on future VMs.

What this project does

This is a standalone Ansible project (originally adapted from a teacher's training repo) that prepares and hardens a fresh Debian/Ubuntu server: installs baseline packages, locks down sshd, configures iptables, fail2ban, auditd, sysctl tuning, log rotation, and installs Lynis for security auditing. It has been tested end-to-end against a local VirtualBox/Vagrant VM.

Repo layout

```
reza-ansible/
├── ansible.cfg
├── .vault.pass
├── preparing.yaml          # entry playbook (preparing_server + docker roles)
├── inventory/
│   ├── host.yaml          # WHICH VM YOU ARE TALKING TO
│   ├── host_vars/
│   │   └── vagrant-test.yaml # per-host overrides (sshd port, allowed users, iptables)
│   └── group_vars/all/
│       └── preparing.yaml  # package list, sshd/iptables/fail2ban defaults, lynis version
├── roles/
│   ├── preparing_server/  # <-- the hardening role
│   │   ├── tasks/
│   │   │   ├── packages.yaml # apt cache + install (NO full upgrade, see notes)
│   │   │   ├── hardening.yaml # sshd, iptables, fail2ban, ssh banner
│   │   │   ├── auditd.yml, fstab.yml, tuning.yml, ...
│   │   └── lynis.yaml       # installs from Debian repo (not CISofy tarball)
│   ├── docker/
│   └── repo-server/
```

1. Before you touch anything

- Have a target VM up and reachable over SSH (Vagrant, or any Debian/Ubuntu box).
- Know the OS: this role assumes **Debian/Ubuntu** (uses apt, and systemd unit names like ssh, iptables, fail2ban). Run `cat /etc/os-release` on the target to confirm before running anything.
- Have community.general installed on your control machine: `ansible-galaxy collection install community.general`
- Keep a **second terminal open with a live SSH session** to the VM before every real (non `--check`) run. This is the single most important habit in this whole workflow — `sshd` and `iptables` tasks can lock you out if something is wrong.

2. Point the project at your VM

2.1 Get the VM's connection details

For a Vagrant VM, run this inside the VM's own folder (where the Vagrantfile lives):

```
vagrant ssh-config
```

Copy down **HostName**, **Port**, **User**, and **IdentityFile** from the output.

2.2 Edit inventory/host.yaml

```
all:
  children:
    repo-servers:
      hosts:
        vagrant-test:
          # <- rename per VM if you like
          ansible_host: 127.0.0.1
          ansible_port: 2222
          ansible_user: vagrant
          ansible_ssh_private_key_file: /path/to/private_key
          ansible_ssh_common_args: '-o StrictHostKeyChecking=no
            -o UserKnownHostsFile=/dev/null
            -o PubkeyAcceptedKeyTypes=+ssh-rsa -o HostKeyAlgorithms=+ssh-rsa'
```

2.3 Edit inventory/host_vars/vagrant-test.yaml

This file must be named to match the host alias used in `host.yaml` (e.g. `vagrant-test.yaml` for host alias `vagrant-test`). It overrides two dangerous defaults from `group_vars`:

```
sshd_config:
  Port: 22                                # the REAL guest ssh port, not the forwarded host
  port
  AllowUsers: "root vagrant"              # must include whichever user you actually connect
  as
  AllowGroups: "root vagrant"
  PermitRootLogin: without-password
  PasswordAuthentication: "yes"

iptables:
  network_adapter_access: [lo]
  tcp_port_access: [80, 443]
  trusted_IP: []
  trusted_Range: [10.0.2.0/24]            # VirtualBox default NAT subnet
```

Why this file exists

group_vars/all/preparing.yaml sets sshd_config.Port from {{ ansible_port }} — the SAME variable used to connect. If ansible_port is the forwarded host port (e.g. 2222), sshd inside the guest would get configured to listen on 2222, but VirtualBox only forwards host:2222 → guest:22. Next connection attempt fails. Likewise, the default AllowUsers is 'root' only — it will lock out any non-root connecting user unless you override it here.

3. Run the playbook

3.1 Dry run first, always

```
cd reza-ansible
ansible-playbook -i inventory/host.yaml preparing.yaml \
  --tags preparing_server --skip-tags external_key \
  --check --diff
```

Read the diff, especially anything touching `sshd_config` and `iptables`, before doing anything for real.

3.2 Run for real

With your second SSH session still open in another terminal:

```
ansible-playbook -i inventory/host.yaml preparing.yaml \
  --tags preparing_server --skip-tags external_key
```

--tags preparing_server runs only the hardening/prep role, skipping the docker role also defined in `preparing.yaml`.

--skip-tags external_key skips a task that fetches an SSH public key from the original teacher's server (`store.dockerme.ir`) — irrelevant, and often unreachable, for a personal test VM.

3.3 Verify afterwards

- Open a **brand new** terminal (don't reuse the safety-net session) and confirm SSH still connects cleanly.
- `sudo lynis show version` — installed tools live in `/usr/sbin`, which is on root's PATH but not a regular user's, so use `sudo` or the full path.
- `systemctl status ssh iptables fail2ban auditd` — all should be active/running.

4. Known gotchas (already fixed in this copy)

These were real failures hit while testing against a Debian 12 (bookworm) Vagrant box. The fixes are already applied in this project, but worth knowing about if you fork this further or hit something similar on a different OS/image.

ntp vs systemd-timesyncd conflict

Debian 12 ships systemd-timesyncd by default. Installing the classic ntp package conflicts with it (both provide time-daemon). Fixed by stopping/removing systemd-timesyncd in packages.yaml before installing the package list.

Full apt upgrade breaking the shared folder

The original 'Update and upgrade apt packages' task ran a full apt-get upgrade, which pulled in a new kernel. VirtualBox's shared-folder driver (vboxsf) isn't rebuilt for a new kernel automatically, so /vagrant stops mounting after the next reboot. Also, installing a new kernel triggers grub-pc's postinstall script, which can hang/fail waiting for a non-interactive answer about which disk to install the bootloader to. Fixed by dropping 'upgrade: yes' — only update the apt cache, don't upgrade the whole system or pull in a new kernel.

ufw / postfix not installed

Minimal Debian/Vagrant boxes often don't ship ufw or postfix at all, so 'stop and disable' tasks for them fail with 'Could not find the requested service'. Fixed with ignore_errors: yes on both tasks — safe since the intent is just 'make sure it's off if present'.

Lynis download blocked (403 Forbidden)

The original task downloaded a tarball directly from cisofy.com/files — that URL now 404s/403s (old version retired, and CISOfy may also geo-restrict certain regions). Fixed by installing lynis from the Debian repo instead (apt install lynis) — simpler, and gets security updates automatically.

authorized_key task (external_key tag)

One hardening task fetches an SSH public key from the original teacher's server (store.dockerme.ir) into root's authorized_keys. Left in place but tagged 'external_key' so it can be skipped — it's not relevant to a personal test VM and the URL may not always be reachable.

5. Using this on a brand-new VM later

The roles and templates don't need to change for a same-OS VM. Only the per-machine identity does:

Step	What to do
1	Get the new VM's SSH host, port, user, and key path.
2	Confirm the OS is Debian/Ubuntu (cat /etc/os-release).
3	Update inventory/host.yaml with the new connection details (rename the host alias if you like).
4	Rename/update inventory/host_vars/<alias>.yaml to match, and sanity-check the sshd_config.Port and AllowUsers
5	Dry-run with --check --diff, read the diff.
6	Run for real with a live second SSH session open.
7	Verify: fresh SSH connection, lynis show version, systemctl status on the key services.

Golden rule

Never run this against sshd/iptables tasks without a second, already-open SSH session to the target. If the sshd_config or iptables rules end up wrong, that live session is the only way back in short of console access.

Generated for the reza-ansible project — standalone copy of a Debian hardening/prep Ansible role, tested against a local VirtualBox/Vagrant VM.